

# THE ML GUIDE FOR EVERY ENGINEER

---

What to learn, where to start, and why it matters  
— based on YOUR type of engineering

By @1nska

NYU Shanghai — 2026

# Why Every Engineer Needs ML

Machine learning isn't just a CS thing anymore. It's becoming the layer that sits on top of every engineering discipline. Mechanical engineers use it to predict equipment failures before they happen. Electrical engineers use it to optimize power grids in real time. Civil engineers use it to model structural integrity under conditions no equation can fully capture.

The engineers who learn ML now won't just be better engineers — they'll be the ones leading projects, starting companies, and solving problems that were previously considered unsolvable.

**This guide is designed to be practical, not academic.** For each engineering discipline, you'll get: why ML matters for your field, what to learn first, which tools to use, and a starter project to build.

# The ML Foundation (For Everyone)

Regardless of your discipline, these fundamentals apply to every engineer getting started with ML. Don't overthink this part — you don't need a PhD in statistics. You need enough to be dangerous.

## Core concepts to understand first:

- Supervised vs. unsupervised learning — know when to use each
- Regression vs. classification — the two most common problem types
- Training, validation, and test data — why you split your data and how
- Overfitting — the #1 mistake beginners make
- Feature engineering — often more important than the model itself

## The learning path (in order):

- **Week 1–2:** Python basics (if you don't already know it). Use Python for Everybody (free) or Codecademy.
- **Week 3–5:** Andrew Ng's Machine Learning Specialization on Coursera. The gold standard intro. Do the assignments.
- **Week 6–8:** Pick ONE project from your discipline section below and build it. Learning by building beats courses every time.
- **Week 9+:** Go deeper into the specific ML subfield that applies to your discipline (see your section).

**Pro tip:** You don't need to master ML theory. You need to know enough to frame your engineering problems as ML problems, choose the right approach, and implement a working solution. That's it.

# Your Discipline-Specific Roadmap

Find your engineering type below. Each section gives you a focused path — no fluff, just what you need.

## Mechanical Engineering

### Why ML matters here:

Predictive maintenance alone is a massive industry. ML models can predict when a machine will fail based on sensor data — vibration, temperature, pressure — saving millions in downtime. Beyond that, generative design uses ML to explore thousands of design variations to find optimal shapes and structures that no human would think of.

### Start with:

- Time-series analysis — your sensor data is sequential, so learn how to work with it
- Regression models — predicting continuous values (temperature, stress, wear)
- Anomaly detection — flagging when something is about to break

### Key tools:

- **scikit-learn** — your first stop for classical ML models
- **TensorFlow / PyTorch** — for deeper neural network models
- **MATLAB ML Toolbox** — if you're already in the MATLAB ecosystem
- **PySpark** — for working with large-scale sensor datasets

**Starter project:** Build a predictive maintenance model using a public bearing vibration dataset (NASA's bearing dataset is free). Train a model to predict remaining useful life of a component.

## Electrical Engineering

### Why ML matters here:

Signal processing and ML are a natural fit. From optimizing power grid loads to designing smarter circuits, ML is already deeply embedded in EE. If you work with signals, images, or any kind of waveform data, neural networks can find patterns your traditional filters miss.

### Start with:

- Signal classification — identifying patterns in waveforms

- Convolutional Neural Networks (CNNs) — not just for images; great for spectrograms and signal data
- Reinforcement learning basics — for control systems and optimization

#### Key tools:

- **PyTorch** — the go-to for signal and research-oriented ML
- **scipy.signal** — preprocessing your signals before feeding to ML
- **Edge Impulse** — deploy ML models on embedded hardware
- **TinyML** — ML that runs on microcontrollers

**Starter project:** Build a signal classifier that can distinguish between different types of electrical faults using publicly available power quality datasets. Deploy it on an Arduino or Raspberry Pi for bonus points.

## Civil / Structural Engineering

#### Why ML matters here:

Infrastructure generates massive amounts of data — strain gauges, traffic sensors, weather patterns, soil tests. ML can process all of it simultaneously to predict structural health, optimize traffic flow, and estimate construction costs more accurately than traditional methods.

#### Start with:

- Regression models — for cost estimation and load prediction
- Computer vision — crack detection, site monitoring, progress tracking
- Geospatial data analysis — combining satellite imagery with ground data

#### Key tools:

- **scikit-learn** — for tabular data (soil data, cost estimates, measurements)
- **OpenCV + YOLO** — for visual inspection and crack detection
- **Google Earth Engine** — for satellite imagery analysis
- **Pandas + GeoPandas** — for geospatial data wrangling

**Starter project:** Train a computer vision model to detect cracks in concrete surfaces using the public SDNET2018 dataset. This is directly applicable to bridge and building inspection.

## Chemical / Process Engineering

### Why ML matters here:

Chemical processes are complex, nonlinear systems with dozens of interacting variables. ML excels at modeling these relationships — predicting reaction yields, optimizing process parameters, and catching quality issues before they become expensive. Pharma and materials science are being transformed by ML-driven discovery.

### Start with:

- Process optimization — finding optimal operating conditions from historical data
- Soft sensors — predicting hard-to-measure variables from easy-to-measure ones
- Molecular property prediction — emerging and high-impact area

### Key tools:

- **scikit-learn** — classical models for process data
- **RDKit** — molecular informatics and cheminformatics
- **DeepChem** — deep learning specifically for chemistry applications
- **Aspen + Python API** — connecting ML to process simulation

**Starter project:** Use a public dataset of chemical reactions (try the USPTO reaction dataset) to build a model that predicts reaction yield based on input conditions. Start with a simple random forest, then try a neural network.

## Biomedical Engineering

### Why ML matters here:

Medical imaging, wearable sensors, genomics — biomedical engineering is one of the fields most transformed by ML. From detecting tumors in X-rays to predicting patient outcomes from wearable data, the applications are immediate and the demand for engineers who can bridge biology and ML is enormous.

### Start with:

- Medical image classification — detecting anomalies in scans
- Time-series from wearables — heart rate, EEG, EMG signal analysis
- Natural Language Processing — extracting insights from clinical notes

### Key tools:

- **PyTorch + MONAI** — purpose-built for medical imaging AI

- **TensorFlow Lite** — deploying models on wearable devices
- **BioPython** — for genomics and bioinformatics
- **Hugging Face** — for NLP models on clinical text

**Starter project:** Build an ECG classification model using the MIT-BIH Arrhythmia Database (free on PhysioNet). Train a model to distinguish normal heartbeats from arrhythmias.

## Software / Computer Engineering

### Why ML matters here:

You might think you already know this — but there's a difference between using ML APIs and understanding how to design ML systems. As a software engineer, your advantage is that you already know how to code and ship products. The gap is in ML system design, model selection, and data pipeline architecture.

### Start with:

- ML system design — how to architect production ML pipelines
- MLOps — model versioning, monitoring, retraining in production
- Deep learning — transformers, LLMs, and their practical applications

### Key tools:

- **PyTorch** — the standard for research and increasingly for production
- **MLflow / Weights & Biases** — experiment tracking and model management
- **Hugging Face** — transformers and pre-trained models
- **LangChain / LlamaIndex** — building applications on top of LLMs

**Starter project:** Build a full ML-powered feature for a product: data pipeline, model training, API endpoint, and monitoring. Use a real dataset, deploy it, and track performance. The full stack experience is what sets you apart.

# Free Resources to Get Started

You don't need to spend money to learn ML. Here are the best free resources, organized by stage.

## Courses

- **Andrew Ng's ML Specialization** (Coursera) — the best starting point, period
- **fast.ai Practical Deep Learning** — top-down approach, build things from day one
- **MIT OpenCourseWare 6.036** — more rigorous, great if you want deeper math foundations
- **Google's ML Crash Course** — quick, practical, excellent for engineers
- **Stanford CS229** (YouTube) — the classic graduate-level course, free on YouTube

## Practice platforms

- **Kaggle** — competitions, datasets, and notebooks. Start with their "Learn" courses
- **Google Colab** — free GPU notebooks. No setup required
- **Papers With Code** — find state-of-the-art models with implementations
- **Hugging Face** — pre-trained models you can fine-tune for your domain

## Books (free online)

- **Hands-On ML with Scikit-Learn, Keras & TensorFlow** by Aurélien Géron — practical and project-driven
- **Deep Learning** by Goodfellow, Bengio & Courville — the deep learning bible (free at [deeplearningbook.org](http://deeplearningbook.org))
- **Pattern Recognition and ML** by Bishop — for those who want the math



# The 8-Week Kickstart Plan

Here's a week-by-week plan to go from zero ML knowledge to having a working project on your resume. Spend 5–8 hours per week. Consistency beats intensity.

|               |                                                                                                                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | <b>Setup &amp; Python review</b><br>Install Python, set up Jupyter/Colab, review NumPy and Pandas basics. If you already know Python, skip to data manipulation exercises.                            |
| <b>Week 2</b> | <b>ML fundamentals</b><br>Start Andrew Ng's course or fast.ai. Focus on understanding supervised learning, linear regression, and classification. Don't get stuck on math — understand the intuition. |
| <b>Week 3</b> | <b>Your first model</b><br>Pick a simple dataset from Kaggle (house prices, titanic). Build a model from scratch using scikit-learn. Submit to a Kaggle competition.                                  |
| <b>Week 4</b> | <b>Deeper models</b><br>Learn decision trees, random forests, and gradient boosting. These are the workhorses of real-world ML. Practice feature engineering.                                         |
| <b>Week 5</b> | <b>Intro to deep learning</b><br>Learn what neural networks are and when to use them. Build a simple image classifier or text classifier using PyTorch or TensorFlow.                                 |
| <b>Week 6</b> | <b>Domain-specific deep dive</b><br>Go to YOUR discipline section in this guide. Start the recommended project. Research how ML is being used in your specific field.                                 |
| <b>Week 7</b> | <b>Build your project</b><br>Dedicate this week to building and iterating on your domain project. Document everything — this becomes your portfolio piece.                                            |
| <b>Week 8</b> | <b>Polish &amp; share</b><br>Clean up your project code, write a README, publish on GitHub. Post about what you built on LinkedIn or Instagram. You now have proof you can do this.                   |

# Now Go Build Something

The best way to learn ML is to apply it to a problem you actually care about. Don't wait until you feel "ready" — you won't. Pick the starter project for your discipline, open a Colab notebook, and start. You'll learn more in one weekend of building than in a month of watching tutorials.

If you want more guides like this — on building startups, switching career paths, or bridging business and engineering — follow me on Instagram. I'm documenting the entire journey.

**Follow @1nska on Instagram** for more guides, build logs, and real talk about building as a student. DM me "PROJECT" and I'll help you pick the right ML starter project for your field.

— Enes (@1nska)  
NYU Shanghai, 2026